

Invoice Ninja → Xero sync — design & decisions

Target org: **Emerald Security LLC** (Xero tenant `d2786ed8-7676-4056-9890-f2b0f62d1132`)
US · USD · calendar financial year · single-member LLC
Goal: a defensible balance sheet and supporting statements for a **Dun & Bradstreet** credit file.

Josh's decision (2026-07-29): clear 2025 out of Xero and rebuild it from Invoice Ninja.

The data, settled

Snapshot v2 (`--lifecycle active+archived`) resolved the earlier contradiction. Invoice Ninja holds **327 invoices** in the window — 318 archived, 9 active, 9 soft-deleted and excluded. v1's `status=active` filter was returning only the 9.

	Invoice Ninja invoiced	Invoice Ninja cash	Xero income
2025 (211 invoices)	\$117,373.75	\$121,681.49	\$27,720.45
2026 YTD (108 invoices)	\$73,140.12	\$72,854.63	\$0.00
Total	\$190,513.87	\$194,536.12	\$27,720.45

Cash exceeds invoiced by \$4,022 over the window — pre-2025 receivables collected in January. Open AR today is \$395 across 5 invoices; 313 of 327 invoices are fully paid. Essentially everything billed has been collected.

2025 revenue missing from Xero: \$89,653.30.

What the 2025 Xero entries actually are

`get_top_customers_by_revenue` for 2025 returns:

Contact	Amount
Stripe	\$25,997.14
Dan Wilson	\$893.52
Venmo	\$628.80
Elk Country RV Resort	\$199.99
Amazon Shopping	\$177.98

Stripe and Venmo are not customers, they are payment rails. Amazon Shopping is a vendor — almost certainly a refund miscoded to income. Combined with AR = \$0.00 and no invoices in Xero, this means Xero's 2025 income is **bank statement lines coded straight to income accounts**, contact name taken from the bank description. 94% of it is a single "Stripe" lump.

That is why the books are short: only Stripe settlements (net of fees) landed in this Xero bank account. Whatever was collected by other means never appeared.

Confirm before acting: in Xero, Accounting → Bank accounts → Account transactions, and check whether the 2025 income rows are Receive Money bank transactions (expected) rather than invoices.

This changes what "clear 2025" means

The instinct is right, the mechanic is not. Those bank lines are **reconciled against real bank statement lines** — real money really did arrive. Deleting them breaks the bank reconciliation and makes Xero's cash stop tying to the bank statement, which is the one number a D&B reviewer is most likely to check.

Re-code, don't delete. The lines stay; what changes is the account they point at.

Recommended structure:

1. **Import Invoice Ninja invoices** as ACCREC → creates the real \$117,374 of 2025 revenue and the AR behind it.
2. **Import Invoice Ninja payments** against those invoices, settling to a new **Stripe Clearing** account (a bank-type account, not the real bank). Invoice Ninja records payments gross — \$121,681 collected against \$117,374 invoiced confirms gross, not net of fees.
3. **Re-code the existing 2025 bank lines** from **Service** income to **Stripe Clearing**. The bank reconciliation is untouched; the income double-count disappears.
4. **The residual balance in Stripe Clearing is Stripe's fees** — post to **Stripe Fees**. Xero currently shows \$0.33 there for all of 2025, which cannot be right against ~\$26K of Stripe volume.

Net effect: revenue becomes \$117,374 from 313 real invoices with real customer names, cash still ties to the bank, and processor fees appear as an expense for the first time.

Deleting the lines instead would give the same P&L and a broken balance sheet.

Before anything is posted — the 2025 tax question

Restating 2025 moves reported revenue from \$27,720 to \$117,374, a difference of \$89,653. If a 2025 return was filed on Xero's figures, it materially understated income and the restated books will not agree with what was filed. That is a conversation with whoever prepared the return, before the rebuild, not after. I am not a tax adviser and this is not tax advice — flagging it because the rebuild produces the discrepancy.

If no return was filed on these numbers, this is simply a correction and the rebuild is uncomplicated.

Scope decisions

Clients: do not import all 1,080. Invoice Ninja holds 1,080 clients (23 active, 1,057 archived) against ~60 that carry invoices. Create Xero contacts only for clients with at least one invoice in scope.

Expenses: out of scope. Invoice Ninja has \$2,357 across the whole window; Xero already booked \$6,769 for 2025 alone from the bank feed. Importing would double-count and still capture a fraction. Three supporting reasons:

- All 43 expenses have a blank `payment_date`, so the agreed settled/unsettled rule would turn every one into an ACCPAY bill and invent \$2,357 of accounts payable.
- Several categories are not P&L expenses — *Owners Draw*, *Personal Expense*, *Payroll - Withholdings*, *Auto - Loan Payments*, *Loan & Interest Payments*. Owner's draw is equity; loan principal is balance sheet.
- 73 vendors against 43 expenses; mostly unused records.

Let the bank feed carry expenses.

Tax: nearly none. Six rates are configured (CA 6%, Arcata 1.5%, Humboldt unincorporated 2.75%, Trinidad 0.75%, Eureka 1.5%, CA+Local 8.75%) but the entire window carries **\$41.48** of tax, all in 2025-06, across **5 inclusive-tax invoices**. Those 5 need `LineAmountTypes: Inclusive`; everything else is `NoTax`.

Drafts excluded. 2 draft invoices worth \$10,637 create no journals in Xero and stay out. Separately worth a look — that is a large amount sitting unsent.

Cancelled/reversed excluded. 5 cancelled, 1 reversed.

Entity mapping

Invoice Ninja	Xero	Notes
Client (<i>with invoices only</i>)	Contact	ContactNumber = IN-C- <id>
Invoice (sent/partial/paid, incl. archived)	ACCREC, AUTHORISED	drives AR
Payment	Payment → Stripe Clearing, not the real bank	see above
Credit	ACCRECCREDIT + allocations	none present in window
Expense	out of scope	
Vendor	—	not needed with expenses out of scope

Rules that are easy to get wrong

1. **DRAFT/SUBMITTED invoices create no journals.** Everything lands **AUTHORISED**.
2. **PAID is not a status you post.** Create **AUTHORISED**, then apply a Payment.
3. **Invoice date and payment date must both be historically accurate**, or AR is wrong at every prior date.
4. **Archived ≠ void.** Invoice Ninja's **status=active** filter drops archived records; they are still real. This is what broke snapshot v1 and it is still live in **in2xero**.
5. **Payments settle to a clearing account, never the fed bank account**, or cash doubles.
6. **Invoices carry no currency_id** — resolves client → group → company settings.
7. **Xero TaxType codes are per-organisation.** Resolve from **GET /TaxRates** at runtime.
8. **AUTHORISED invoices can only be voided, never deleted.** Rehearse on the Xero Demo Company, which can be reset.

Volume and rate limits

319 invoices + 319 payments + ~60 contacts. Batched at 50 per call that is roughly **16 API calls**; unbatched about 700, which fits Xero Starter's 1,000/day but leaves no room to retry. Batch.

Safety design

- **Nothing posts unless it reconciles** — each document rebuilt line by line against Invoice Ninja's own **amount**; sub-cent drift gets a rounding line, larger drift is refused and flagged.
- **Deterministic Idempotency-Key** per source record.

- **Resumable**, with the SQLite crosswalk as checkpoint; client watches `X-DayLimit-Remaining`.
- **Watermarks record run start, not run end.**

Auth

Xero Custom Connection (`client_credentials`, US\$5/month, one org). No refresh token, no 60-day cliff — the auth-code flow's rotating refresh token can brick an unattended cron job. Both flows implemented.

Open questions

1. **Was a 2025 return filed on Xero's \$27,720?** Blocking — see above.
2. Confirm the 2025 income rows are bank transactions, not invoices.
3. What stopped the feed in January 2026, and how does 2026 cash get caught up?
4. Chart of accounts *codes* — names known, codes not. Accounting → Advanced → Chart of accounts.

Tooling

in2xero — ~2,350 lines incl. tests. **Needs three changes before any real run:** the archived-record bug (same as snapshot v1), payments routed to a clearing account, and expenses dropped from scope.

ninja_snapshot.py v2 — delivered and run successfully 2026-07-29.